

CNNs for Image Classification

Shavin Kaluthantri
The University of Adelaide
a1904121@adelaide.edu.au

November 1, 2023

Abstract

In an attempt to develop an accurate traffic sign classifier, multiple architectures were explored ranging from base Convolution Neural Network (CNN) to more sophisticated ones like VGG16 and ResNet34. These architectures were initially trained with a set of standard hyperparameters and then optimized through a random search over different combinations of hyperparameters. Data augmentation played a pivotal role, and its efficacy was clearly evidenced by the performance leap in the base CNN model when augmented data was used. The VGG16 model trained on augmented data exhibited remarkable performance, attaining a validation accuracy nearing 97%. However, through rigorous hyperparameter tuning, an even higher accuracy of 98.76% was achieved using the SGD optimizer. The final model, incorporating the best hyperparameters and a learning rate scheduler, achieved an impressive validation accuracy of 99.51% and 99.68% on the test dataset.

1 Introduction

Image classification serves as a pivotal cornerstone in the domain of computer vision and machine learning, boasting applications spanning from autonomous driving to medical diagnostics. With the surge in computational capabilities and the availability of expansive datasets, the proficiency of machines in recognizing and classifying images has witnessed significant breakthroughs. Among the diverse architectures deployed for image classification, CNNs stand out

as a seminal architecture. Their hierarchical structure is naturally suited to recognize increasingly complex patterns in visual data, making them particularly effective for image recognition tasks [4]. This project specifically focuses on the German Traffic Sign Recognition Benchmark (GTSRB) dataset, addressing the crucial challenge of recognizing and categorizing traffic signs, a problem intrinsically tied to the efficacy and safety of autonomous driving systems [3].

A thorough comparative analysis was performed with other state-of-the-art algorithms to benchmark the effectiveness of the proposed method with leading competitors. Metrics such as Area Under the Curve (AUC) and accuracy are used to provide a robust assessment of predictive accuracy.

Challenges like illumination variations in image classification have been tackled with unique architectures like the Separating-Illumination Network (Sill-Net), which focuses on removing the semantic part of images and optimizing with illumination features [10].

Addressing the computational challenges of deploying deep neural networks in embedded systems, a novel architecture, MicronNet, has been introduced. MicronNet, designed on microarchitecture principles, provides a compact solution with just approximately 1MB model size and around 510,000 parameters - which is about 27 times fewer parameters than contemporary models. Yet, it achieves a remarkable 98.9% top-1 accuracy on the GTSRB dataset. The compactness of MicronNet doesn't compromise its speed, requiring just around 32.19 ms for inference

on efficient processors, making it an ideal candidate for embedded traffic sign recognition scenarios [9]

Recently, a Deep Learning approach has emerged, combining the power of CNNs with Spatial Transformer Networks for traffic sign recognition [1]. Through rigorous experimentation on traffic sign datasets from Germany and Belgium, this approach has evaluated various adaptive and non-adaptive stochastic gradient descent optimisation algorithms and configurations of Spatial Transformer Networks within the CNN architecture. Remarkably, this method has outperformed existing state-of-the-art algorithms by achieving an impressive 99.71% accuracy rate on the GTSRB dataset without the need for hand-crafted data augmentation techniques and with fewer memory requirements [1]. Such advancements bolster the promise and capability of CNNs in the domain of traffic sign recognition.

This project focuses on the use of several CNNs to evaluate the best architecture that leads to the highest accuracy on the GTSRB dataset. Initially, a simple CNN was built from scratch to serve as a baseline model for the comparative analysis. Next, the dataset was trained on the following architectures: Visual Geometry Group Networks (VGGNet) and Residual Networks (ResNet) to evaluate the performance of the dataset on deeper and more complex architectures.

Data augmentation techniques were used to increase the training dataset with transformed images and the best-chosen architecture was fine-tuned using hyperparameter tuning to obtain the parameters that lead to the highest accuracy. Finally, these parameters were used along with several optimization techniques to obtain a refined model with significantly improved accuracies with a noteworthy performance on the GTSRB dataset.

2 Methodology

2.1 Data Preparation

The German Traffic Sign Recognition Benchmark (GTSRB) is a widely used dataset in the realm of traffic sign recognition. Developed by the Insti-



Figure 1: Random selection of the 43 classes in the GTSRB dataset.

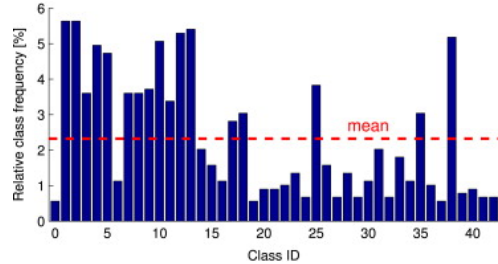


Figure 2: Relative frequencies of classes

tute for Neural Computation at the Ruhr-University Bochum, Germany, the dataset serves as a benchmark for evaluating and comparing different techniques in traffic sign recognition [3].

GTSRB encompasses over 50,000 images covering 43 distinct traffic sign classes. These images are real-world traffic signs captured under various conditions, which makes the dataset comprehensive and challenging. The images come in different sizes and are presented in varied lighting conditions, angles, and occlusions [8]. Such variety is essential as it mirrors real-world scenarios where signs can be partially hidden, faded, or damaged.

A randomly selected image from each class is shown in Fig. 1 and the corresponding frequencies of the class ID are shown in Fig. 2. Fig. 3 shows the distribution of traffic sign sizes, taking into account the larger of both dimensions of the traffic sign ROI [8].

The dataset was resized to 32 x 32 pixels since the highest frequency of images was in that region. This size also provides a good balance between computa-

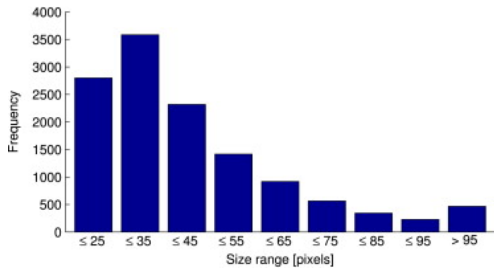


Figure 3: Distribution of size of images

tional efficiency and model performance.

The dataset is then partitioned and initially split into train and test sets with a split of 80:20. The train set is further split by a ratio of 80:20 to account for the train and validation sets respectively. The training set is employed to train the model and the validation set is used to evaluate the performance of the model and to tune the hyperparameters while the test set is used to evaluate the final model. Notably, the test and validation sets have been kept separate from the training set to ensure an unbiased evaluation of the model’s true capabilities. Each image in the dataset is annotated with information about its class which aids in accurate model training and validation.

The mean and standard deviation values for the Red Green, and Blue (RGB) channels of the images were computed using the training dataset and these values were used to normalize the training, validation, and test datasets. Normalization allows the neural network to converge faster reducing the number of iterations needed to train the model.

2.1.1 Data Augmentation

To enhance the accuracy of the models on the GT-SRB dataset, several data augmentation techniques were employed. Data augmentation is a widely accepted strategy in deep learning, especially for computer vision tasks, as it introduces variability into the training dataset. By doing so, it helps prevent overfitting and enables the model to generalize better to unseen data. Below is a detailed breakdown of the data augmentation steps taken:

RandomHorizontalFlip(): This augmentation ran-

domly flips an image along its horizontal axis. By introducing such random horizontal flips, the model is trained to recognize and classify images that may appear in mirror-like orientations, thus enhancing its robustness. RandomRotation(10): This step introduces a random rotation to the input image, with angles limited to a range of ± 10 degrees. Rotations can simulate various orientations the traffic signs might be captured in real-world scenarios, ensuring that slight tilts or misalignments in the sign orientation do not hinder the model’s performance. ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2): This augmentation varies the brightness, contrast, and saturation of the images. Specifically, the brightness, contrast, and saturation of the image are adjusted by factors randomly sampled from the range $[0.8, 1.2]$. This ensures the model becomes invariant to minor changes in lighting conditions or color variations, which is critical since real-world images can be affected by different weather conditions, camera qualities, and lighting conditions. RandomAffine(degrees=0, translate=(0.1, 0.1)): The RandomAffine transformation applies an affine transformation to the image. In this specific case, no rotation (degrees=0) is applied; however, translations up to 10% (translate=(0.1, 0.1)) of the image’s width and height are allowed. This simulates slight shifts or off-center captures of the traffic signs, ensuring the model learns to recognize signs even if they aren’t perfectly centered.

Incorporating these augmentation techniques aims to create a more diverse training dataset, allowing the model to witness various possible occurrences of the data. As a result, the trained models are anticipated to demonstrate enhanced performance and robustness when tested on the GTSRB dataset.

2.2 Model Construction

Three CNN architectures were considered for this analysis:

2.2.1 Base CNN Architecture

The BaseCNN architecture, shown in Tab. 1, is a convolutional neural network (CNN) designed for the

Table 1: BaseCNN Architecture

Layer	Configuration
conv1	Conv2d(3, 16, kernel_size=(5, 5), stride=(1, 1))
pool	MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
conv2	Conv2d(16, 32, kernel_size=(5, 5), stride=(1, 1))
fc1	Linear(in_features=800, out_features=120, bias=True)
fc2	Linear(in_features=120, out_features=84, bias=True)
fc3	Linear(in_features=84, out_features=43, bias=True)

task of traffic sign recognition. It integrates various layers, each with distinct functions, to process and transform input images into a format from which the model can infer the corresponding traffic sign class. The architecture’s design draws inspiration from pioneering works on CNNs for image recognition tasks [4, 5].

1. Convolutional Layers:

conv1: The first convolutional layer takes in an input image with 3 channels (typically corresponding to the RGB channels). It utilizes 16 filters, each of size 5x5, to extract low-level features from the image such as edges and corners. The choice of a 5x5 kernel size is effective for capturing more global information in each convolution operation [7]. conv2: The second convolutional layer, with 32 filters of size 5x5, further processes the output from the previous layer. This layer is designed to recognize more complex features, building on the elementary features detected by the first layer.

2. Pooling Layer:

pool: Following each convolutional layer, a max-pooling operation is applied with a 2x2 window and a stride of 2. This reduces the spatial dimensions of the feature maps while retaining the most salient information, effectively rendering the model more translationally invariant and reducing computational de-

mands [5].

3. Fully Connected Layers:

fc1: This is the first fully connected layer, which transforms the feature maps from the previous layers into a flattened 1D vector. It consists of 120 neurons and serves as a bridge between the convolutional layers and the final classification layers, capturing high-level features in the image. fc2: The second fully connected layer, consisting of 84 neurons, further processes the outputs from the preceding layer, ensuring that the network can recognize intricate patterns and relationships in the data. fc3: The final fully connected layer has 43 neurons, corresponding to the number of traffic sign classes. It outputs the probability distribution over the 43 classes, enabling classification.

4. Activation Function:

The ReLU (Rectified Linear Unit) activation function is employed after each convolutional and fully connected layer (except the final layer). ReLU introduces non-linearity to the model, enhancing its capacity to capture complex patterns in the data [6]. In terms of the network’s forward propagation: The input image undergoes convolution operations, followed by ReLU activation and max-pooling. After traversing through the convolutional and pooling layers, the feature maps are flattened and passed through the fully connected layers. The final output provides the likelihood for each traffic sign class, from which the class with the highest probability is typically selected as the prediction.

2.2.2 VGG Architecture



Figure 4: Architecture of VGG16

The VGG16 architecture, developed by the Visual Geometry Group at Oxford, is one of the seminal deep learning models introduced in the context of the

ILSVRC-2014 competition, where deep convolutional neural networks were benchmarked on the ImageNet dataset [7].

Architecture Details: **Input Layer:** As shown in Fig. 4, the input to the VGG16 model is a fixed-size 224 x 224 RGB image. **Convolutional Layers:** The architecture contains 13 convolutional layers. These are organized into 5 blocks, with each block consisting of 2-3 convolutional layers. In the first two blocks, the convolutional layers use 64 and 128 filters respectively, of size 3x3 and stride 1. The next three blocks use 256, 512, and 512 filters respectively. **Activation Function:** After each convolutional layer, the Rectified Linear Unit (ReLU) activation function is applied, introducing non-linearity [6]. **Pooling Layers:** After each block of convolutional layers, a max-pooling layer is added. The pooling operation has a 2x2 receptive field and a stride of 2, which effectively downsamples the spatial dimensions of the feature maps by half. **Fully Connected Layers:** Following the convolutional and max-pooling layers, there are 3 fully connected layers. The first two fully connected layers have 4096 nodes each. The third and final fully connected layer originally produces a 1000-dimensional output, corresponding to the 1000 ImageNet classes. However, this layer has been modified to output 43 dimensions, to match the number of classes GTSRB dataset.

2.2.3 ResNet Architecture

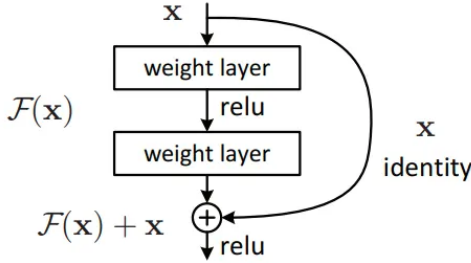


Figure 5: Skip connections in Resnet

ResNet, which stands for "Residual Networks," was introduced by He et al. in their 2015 paper [2].

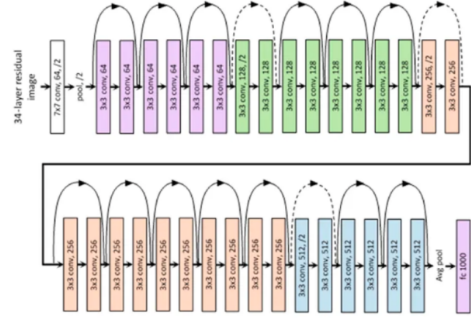


Figure 6: Architecture of ResNet34

The fundamental insight behind ResNet is the use of residual connections (or skip connections) as shown in Fig. 5 that bypass one or more layers. This architecture was proposed to solve the vanishing gradient problem, enabling the training of very deep networks.

Initial Convolutional and Pooling Layers: As shown in Fig. 6, the input undergoes an initial convolution with 64 filters several and stride 2. This is followed by the ReLU activation function. Then, there's a max-pooling layer with a 3x3 filter size and stride of 2. **Residual Blocks:** ResNet-34 contains multiple "blocks" of layers. Each block has several "residual units." The defining characteristic of these blocks is the residual connection that skips one or more layers. **Detailed Layer Blocks:** Block 1: 3 residual units with 64 filters of size 3x3. Block 2: 4 residual units with 128 filters of size 3x3. Note: The first unit reduces the spatial dimensions by half using a stride of 2. Block 3: 6 residual units with 256 filters of size 3x3. The first unit also uses a stride of 2. Block 4: 3 residual units with 512 filters of size 3x3. As before, the first unit has a stride of 2. **Global Average Pooling and Fully Connected Layer:** After the residual blocks, there's a global average pooling layer which reduces the spatial dimensions to 1x1. This is followed by a fully connected layer with a number of neurons equal to the number of classes in the dataset. This was initially set to 1000 (for ImageNet) but was adjusted to 43 to match the number of classes in the GTSRB dataset.

2.3 Hyperparameter Tuning

2.3.1 Random Search

Once the best-performing architecture was chosen, the model from the chosen architecture was optimized by performing a random search over a hyperparameter space, enumerating possible values for batch size, learning rate, momentum, optimizer type, and weight decay. Ten random combinations of these hyperparameters were then sampled. For each combination, the model’s architecture was defined and trained with the given hyperparameters. The validation accuracy for each combination was then recorded. After iterating through all sampled hyperparameter combinations, the one yielding the highest validation accuracy was chosen as the optimal set of parameters to be used as the final model.

2.3.2 Early stopping

The EarlyStopping class was created to facilitate the early termination of training if the validation loss does not improve beyond a predefined minimum delta after a certain number of epochs, denoted as patience. This ensures that the model doesn’t overfit and reduces computational costs by halting unnecessary training.

2.3.3 Fine tuning

The best combination of hyperparameters was used to train the final model with a higher number of iterations (epochs) so that the model converges with a higher accuracy. A learning rate scheduler ‘LinearLR’ has been used to gradually reduce the learning rate, which helps in achieving a more stable convergence and fine-tuning the model towards the end of training. The learning rate scheduler adjusts the learning rate of the given optimizer linearly starting at 100% of the originally set learning rate in the optimizer. By the end of the specified iterations, the learning rate would be reduced to 50% (or 0.5 times) of its original value.

3 Results

In the initial experimentation stage, the following variables were kept constant:

- Learning rate: 0.001
- Epochs: 5
- Batch Size: 64
- Loss function: CrossEntropyLoss, suitable for classification tasks where classes are mutually exclusive.
- Optimiser: Stochastic Gradient Descent (SGD) with a learning rate of 0.001 and momentum of 0.9.

Momentum prevents the optimizer from getting stuck in local minima and speeds up convergence [?]. The images of the GTSRB dataset were resized to a size of (224 x 224) to match the input size of VGG16 and ResNet34 architectures. The models are then compiled and trained using the training dataset.

The validation accuracies and loss for the proposed architectures have been summarised in Tab. 2

Model	Data Aug	Accuracy	Loss
base_cnn	No	0.468	1.95727
base_cnn_aug	Yes	0.6798	1.2066
vgg16	Yes	0.9704	0.0973
resnet34	Yes	0.8820	0.333

Table 2: Accuracies for different architectures

The model with VGG16 architecture trained on the augmented dataset was the best-performing model in the initial model selection phase. This model obtained an accuracy of 0.9704 on the validation dataset.

The results of the ten chosen models obtained from random search across hyperparameter space are shown in Tab. 3 and Tab. 4. Tab. 3 uses SGD optimizer with a range of momentum values while Tab. 4 uses ADAM optimizer with a range of values for its weight decay parameter. The parameters of the best performing model have been highlighted.

BS	LR	momentum	opt	val_acc
32	0.0001	0.99	SGD	0.9568
16	0.001	0.95	SGD	0.9876
32	0.0001	0.9	SGD	0.2228

Table 3: Hyperparameter Performance (SGD)

BS	LR	opt	w_decay	val_acc
32	0.001	Adam	0.0	0.0568
128	0.0005	Adam	0.0	0.9829
32	5e-05	Adam	0.0	0.9833
128	0.0005	Adam	0.0001	0.0554
64	0.0005	Adam	0.0001	0.9871
16	0.001	Adam	0.0	0.0568
32	5e-05	Adam	0.0001	0.9801

Table 4: Hyperparameter Performance (Adam)

Overall, the model with the best accuracy was obtained using the SGD optimizer with a momentum of 0.95, a learning rate of 0.01, and a batch size of 16. The model obtained an accuracy of 0.9876. Several models that used Adam Optimiser had similar but slightly lower accuracies.

The final model used the best hyperparameters and was trained for 10 Epochs, the learning rate was reduced gradually to using the learning rate scheduler. This model obtained a validation accuracy of 0.9951 with a validation Loss of 0.0219. This model obtained an accuracy of 0.9968 on the test dataset. The accuracy curves and loss curves are shown in Fig. 7

4 Discussion

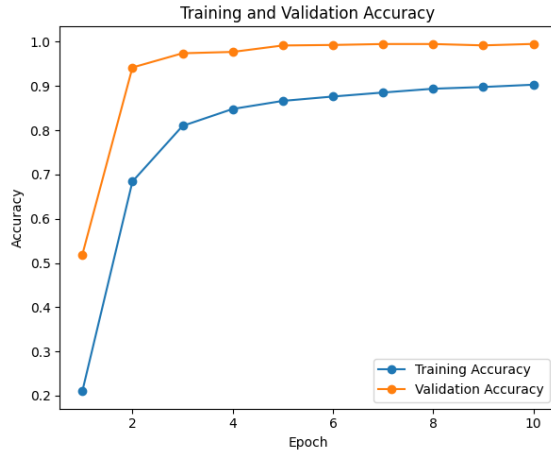
The model based on the VGG16 architecture and trained on augmented data indisputably outperformed the other models, with an accuracy nearing 97%. Such a high accuracy hints at VGG16’s suitability for this specific dataset and task. One plausible reason for VGG16’s superior performance might be its architectural depth, characterized by multiple layers that can capture intricate patterns in images, making it adept at handling complex classification

tasks like the one in question.

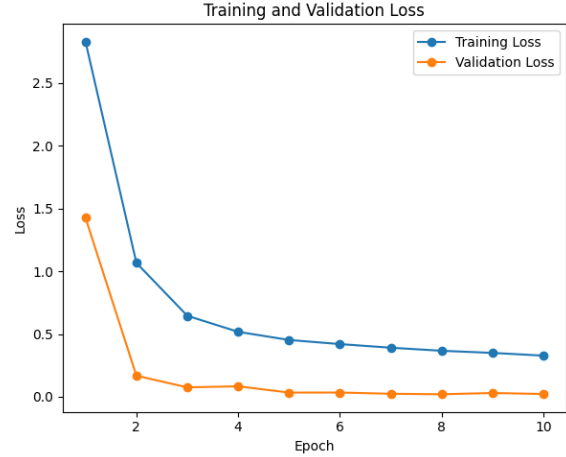
On the other hand, the performance of the base CNN model, especially without data augmentation, serves as a testament to the effectiveness of more sophisticated architectures and the benefits of data augmentation. The improved performance of `base_cnn_aug` compared to `base_cnn` highlights the role of data augmentation in enhancing the model’s ability to generalize, thereby leading to better validation accuracy

Furthermore, the exploration of hyperparameters across two different optimizers, SGD and Adam, provides deeper insights. From Tab. 4 and Tab. 3, it’s clear that the model using the SGD optimizer, with specific settings, emerged as the best with a validation accuracy of 98.76%. While several models using the Adam optimizer showed commendable performance, they couldn’t surpass the best-performing model under SGD. This suggests that, for this specific problem and dataset, SGD’s method of optimizing, when tuned with the right hyperparameters, can potentially yield superior results compared to Adam. This outcome highlights the significance of hyperparameter tuning in machine learning, emphasizing the importance of selecting the right optimiser for the task in hand.

As shown in Fig. 7 the final model was able converge better with a larger epochs and use of a learning rate scheduler. This can be seen by the slight improvements in accuracy in the final few epochs. The training accuracy of final model is lower than its validation accuracy which denotes that model performs better on the validation dataset when compared to the training dataset. This unusual behaviour is mainly due to data augmentation. The data augmentation was performed on the input dataset, and transformed images were added to the input dataset doubling the size of the input dataset fed to the model. This introduces variability and more difficult examples into the training data by transforming the original images. This can make the training data more challenging than the original validation data, leading to lower training accuracy. Therefore, the augmented data provides a richer set of examples that can make the model more robust and generalize better, which improves performance on the validation set. The per-



(a) Accuracy plots



(b) Loss plots

Figure 7: Training and validation accuracy/loss plots

formance of the final model is re-validated by the high accuracy of 99.51% on the test dataset.

5 Code

The code can be accessed here:

https://github.com/shavinkalu23/7318_assignment2

6 Conclusion

The comprehensive experimentation conducted in this study revealed that while basic CNN architectures can benefit considerably from data augmentation, architectures like VGG16 inherently possess the capability to outshine others in intricate classification tasks. A critical takeaway is the paramount importance of hyperparameter tuning. The results favored the SGD optimizer for this specific dataset when configured with the right settings, overshadowing the performance of the often-preferred Adam optimizer. The final model’s learning curve illustrated the benefits of a learning rate scheduler, which assisted in achieving fine-tuned convergence, especially in the terminal epochs. A notable observation was

the model’s lower training accuracy compared to validation, which can be attributed to the increased complexity introduced by data augmentation. Future endeavors can probe into different data augmentation techniques or potentially combine multiple architectures for ensemble predictions. Another prospective avenue could be exploring the impact of other Spatial transformer networks with use of dropout and other regularisation techniques.

References

- [1] Deep neural network for traffic sign recognition systems: An analysis of spatial transformers and stochastic optimisation methods. *Neural Networks*, 99:158–165, 2018. 2
- [2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. 5
- [3] Sebastian Houben, Johannes Stallkamp, Jan Salmen, Marc Schlipsing, and Christian Igel. Detection of traffic signs in real-world images: The German Traffic Sign Detection Benchmark. In *International Joint Conference on Neural Networks*, number 1288, 2013. 1, 2

- [4] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25, 2012. 1, 4
- [5] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. 4
- [6] Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th international conference on machine learning (ICML-10)*, pages 807–814, 2010. 4, 5
- [7] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014. 4, 5
- [8] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel. Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition. *Neural Networks*, (0):–, 2012. 2
- [9] Alexander Wong, Mohammad Javad Shafiee, and Michael St Jules. Micronnet: A highly compact deep convolutional neural network architecture for real-time embedded traffic sign classification. *IEEE Access*, 6:59803–59810, 2018. 2
- [10] Haipeng Zhang, Zhong Cao, Ziang Yan, and Changshui Zhang. Sill-net: Feature augmentation with separated illumination representation. *arXiv preprint arXiv:2102.03539*, 2021. 1